

Placement Intelligence Platform

Master Build Blueprint & Resumable Implementation Guide

Version 1.0

Date: June 18, 2026

Author: AI Architect + Sumit Kumar Singh

IMPORTANT: This document is the single source of truth for building the Placement Intelligence Platform. Feed this document at the start of every session to resume from where you left off. The Progress Tracker section at the end tracks exact completion status of every task.

Table of Contents

- > 1. Project Overview & Context
- > 2. Architecture Overview
- > 3. Technical Stack (Exact Versions)
- > 4. Project Directory Structure (Complete File Tree)
- > 5. Database Schema (All 15+ Tables)
- > 6. Module 1: Authentication System
- > 7. Module 2: Student Dashboard
- > 8. Module 3: Interview Experience Collection
- > 9. Module 4: Company Intelligence Engine
- > 10. Module 5: Resume Intelligence
- > 11. Module 6: AI Mock Interview System
- > 12. Module 7: Voice Interview Engine
- > 13. Module 8: Evaluation Engine
- > 14. Module 9: Interview Replay
- > 15. Module 10: Weakness Detection Engine
- > 16. Module 11: Learning Recommendations
- > 17. Module 12: Placement Readiness Engine
- > 18. Module 13: Knowledge Base Search
- > 19. API Contract (All Endpoints)
- > 20. Frontend Pages & Component Tree
- > 21. AI Service Architecture
- > 22. Environment & Deployment Guide
- > 23. Build Order & Dependency Graph
- > 24. Session Resumption Protocol
- > 25. Progress Tracker (Master Checklist)

Section 1: Project Overview & Context

1.1 What is This?

The Placement Intelligence Platform is a continuously learning placement preparation ecosystem. It is NOT a generic AI mock interviewer. It collects real interview experiences from students, builds company-specific intelligence, conducts AI-powered mock interviews that adapt based on the student's resume and target companies, tracks weaknesses over time, and provides personalized placement readiness analytics.

1.2 Core Philosophy

The moat is NOT AI. The moat is:

- > Real interview experiences submitted by students
- > Company-specific intelligence derived from aggregated data
- > Historical placement data and trending patterns
- > Student performance trends tracked over time
- > Continuously growing knowledge base that improves with every submission

Every interview submitted must improve the platform for all future users.

1.3 User Roles

Student (MVP)

- > Login/Register
- > Upload resume
- > View dashboard with stats & trends
- > Submit interview experiences
- > Explore company intelligence
- > Take AI mock interviews (text + voice)
- > View interview history & replay
- > View performance trends & weakness analysis
- > View personalized recommendations
- > Check placement readiness scores

Admin (MVP)

- > Manage companies
- > Manage/moderate interview experiences
- > Approve flagged content
- > Monitor platform health
- > View analytics dashboard

Mentor (MVP)

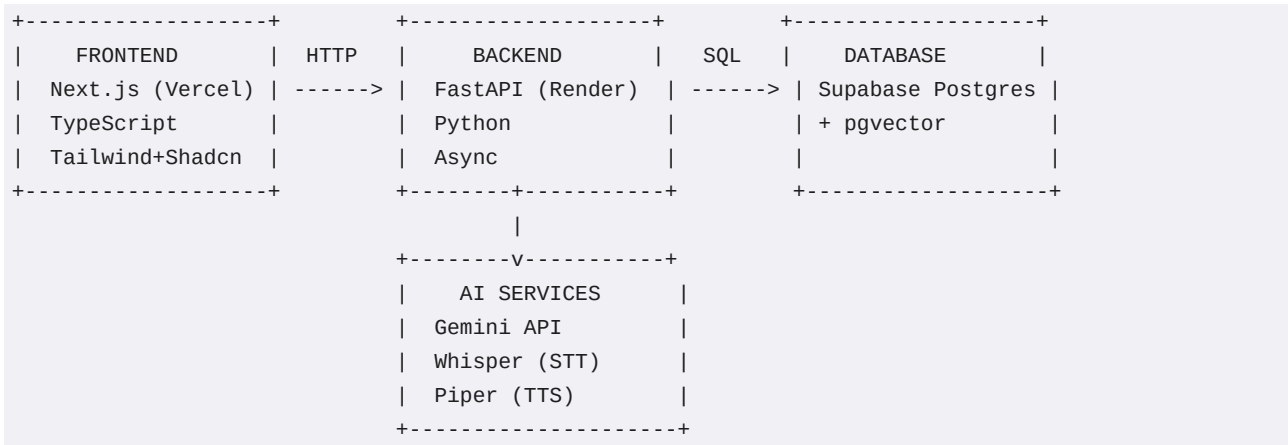
- > View student performance
- > Analyze cohort weaknesses

> Track readiness metrics

NOTE: Mentor role: Design DB tables now but do NOT implement API/UI in MVP.

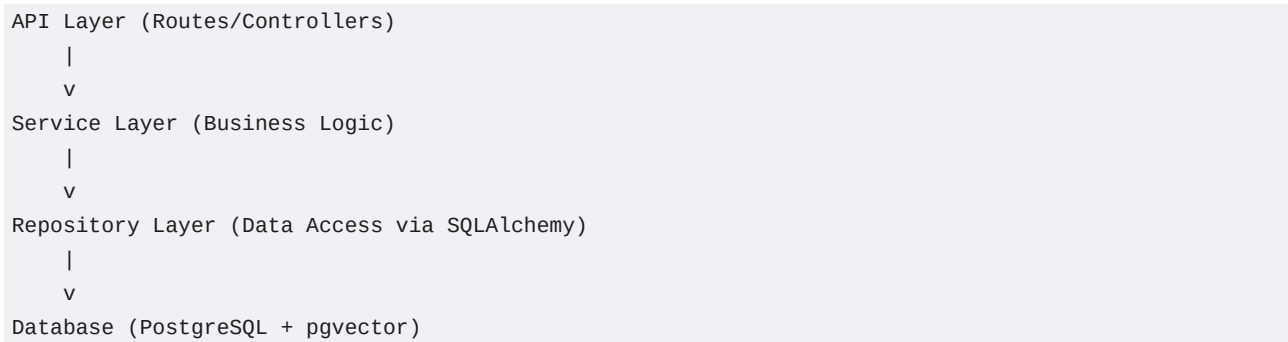
Section 2: Architecture Overview

2.1 High-Level Architecture



2.2 Backend Architecture Pattern

Clean Architecture with Domain-Driven Design:



2.3 AI Service Architecture

All AI services are modular and independently replaceable:

Service	Responsibility
ResumeParserService	Extracts skills, projects, experience from resume PDF/text
InterviewGeneratorService	Generates interview questions based on resume + company + history
FollowUpGeneratorService	Generates adaptive follow-up questions based on previous answers
EvaluatorService	Scores answers with explainable reasoning across 6 dimensions
RecommendationService	Generates personalized study plans and practice recommendations
KnowledgeExtractorService	Extracts structured metadata from raw interview submissions
AnalyticsEngine	Computes company intelligence, readiness scores, weakness patterns

NOTE: Each AI service has a base interface. Swap Gemini for Ollama/OpenAI without touching business logic.

Section 3: Technical Stack (Exact Versions)

3.1 Frontend

Technology	Version	Purpose
Next.js	15.x	React framework with App Router
TypeScript	5.x	Type safety
Tailwind CSS	3.4.x	Utility-first styling
Shadcn UI	latest	Component library (copy-paste)
Zustand	4.x	Client state management
TanStack Query	5.x	Server state / data fetching
React Hook Form	7.x	Form management
Zod	3.x	Schema validation
Recharts	2.x	Charts and trend graphs
Lucide React	latest	Icons

3.2 Backend

Technology	Version	Purpose
Python	3.11+	Runtime
FastAPI	0.115.x	Async web framework
SQLAlchemy	2.0.x	Async ORM
Alembic	1.13.x	Database migrations
asyncpg	0.29.x	Async PostgreSQL driver
Pydantic	2.9.x	Data validation & schemas
python-jose	3.3.x	JWT handling
passlib	1.7.x	Password hashing (bcrypt)
google-genai	0.8.x	Gemini AI integration
pgvector	0.3.x	Vector similarity search
openai-whisper	latest	Speech-to-text
Celery	5.4.x	Async task processing
Redis	5.1.x	Celery broker + caching

3.3 Infrastructure

Service	Details
Database	Supabase PostgreSQL + pgvector extension
Auth	Supabase Auth (Email + Google OAuth)
Frontend Deploy	Vercel (auto-deploy from Git)
Backend Deploy	Render or Railway (Docker)

File Storage	Supabase Storage (resumes, audio)
Task Queue	Celery + Redis (AI processing, async jobs)

Section 4: Project Directory Structure

4.1 Root Structure

```
placement-intelligence-platform/
|-- backend/           # FastAPI Backend
|-- frontend/         # Next.js Frontend
|-- docs/             # Documentation
|-- docker-compose.yml # Local dev environment
|-- README.md         # Project readme
|-- PLAN.pdf          # THIS DOCUMENT
```

4.2 Backend File Tree

```
backend/
|-- app/
|   |-- __init__.py
|   |-- main.py           # FastAPI app entry point
|   |-- core/
|       |-- __init__.py
|       |-- config.py     # Pydantic Settings
|       |-- security.py   # JWT, password hashing, deps
|   |-- db/
|       |-- __init__.py
|       |-- session.py    # Async engine + session
|   |-- models/
|       |-- __init__.py   # All model imports
|       |-- user.py       # User, Profile, Role, Permission, UserRole
|       |-- company.py    # Company, CompanyAnalytics
|       |-- interview_experience.py # InterviewExperience, Question, Outcome
|       |-- mock_interview.py # MockInterview, MockQuestion, Evaluation
|       |-- intelligence.py # Weakness, Recommendation, ResumeData, ActivityLog
|   |-- schemas/
|       |-- __init__.py
|       |-- auth.py       # Login, Register, Token schemas
|       |-- user.py       # Profile update, user response
|       |-- company.py    # Company CRUD schemas
|       |-- interview_experience.py # Submission + response schemas
|       |-- mock_interview.py # Mock interview schemas
|       |-- evaluation.py # Evaluation response schemas
|       |-- intelligence.py # Weakness, Recommendation, Readiness
|       |-- dashboard.py  # Dashboard aggregated data
|   |-- api/
|       |-- __init__.py
|       |-- router.py     # Main API router
|       |-- routes/
|           |-- __init__.py
|           |-- auth.py    # POST /auth/register, /auth/login, /auth/google
|           |-- users.py   # GET/PUT /users/me, /users/me/profile
|           |-- dashboard.py # GET /dashboard
|           |-- companies.py # CRUD /companies
|           |-- interview_experiences.py # CRUD /experiences
```



```
| | | |-- mock_interviews.py # POST /interviews/start, /interviews/{id}/answer
| | | |-- evaluations.py # GET /evaluations/{interview_id}
| | | |-- resume.py # POST /resume/upload, GET /resume/insights
| | | |-- recommendations.py # GET /recommendations
| | | |-- readiness.py # GET /readiness, /readiness/{company_id}
| | | |-- knowledge_base.py # GET /search
| | | |-- voice.py # POST /voice/transcribe, /voice/synthesize
| | | |-- admin.py # Admin-only routes
|-- services/
| | |-- __init__.py
| | |-- auth_service.py
| | |-- user_service.py
| | |-- company_service.py
| | |-- experience_service.py
| | |-- dashboard_service.py
| | |-- mock_interview_service.py
| | |-- recommendation_service.py
| | |-- readiness_service.py
| | |-- knowledge_base_service.py
| | |-- ai/
| | | |-- __init__.py
| | | |-- base.py # Abstract AI service interface
| | | |-- gemini_client.py # Gemini API wrapper
| | | |-- resume_parser.py # Resume parsing + extraction
| | | |-- interview_generator.py # Question generation
| | | |-- followup_generator.py # Adaptive follow-up logic
| | | |-- evaluator.py # Explainable scoring
| | | |-- knowledge_extractor.py # Metadata extraction from submissions
| | | |-- recommendation_engine.py # Personalized recommendations
| | | |-- voice_engine.py # Whisper + Piper integration
|-- utils/
| | |-- __init__.py
| | |-- helpers.py
| | |-- constants.py
-- alembic/
| |-- env.py
| |-- versions/ # Migration files
-- alembic.ini
-- requirements.txt
-- Dockerfile
-- .env.example
```

4.3 Frontend File Tree

```
frontend/  
|-- src/  
|   |-- app/  
|       |-- layout.tsx           # Root layout  
|       |-- page.tsx            # Landing page  
|       |-- globals.css         # Tailwind imports  
|       |-- (auth)/  
|           |-- login/page.tsx  
|           |-- register/page.tsx  
|           |-- (dashboard)/  
|               |-- layout.tsx   # Authenticated layout with sidebar
```

```

| | | |-- dashboard/page.tsx      # Student dashboard
| | | |-- profile/page.tsx        # Profile management
| | | |-- experiences/
| | | | |-- page.tsx              # List experiences
| | | | |-- new/page.tsx          # Submit new experience
| | | | |-- [id]/page.tsx         # Experience detail
| | | |-- companies/
| | | | |-- page.tsx              # Company list
| | | | |-- [id]/page.tsx         # Company intelligence page
| | | |-- interviews/
| | | | |-- page.tsx              # Interview history
| | | | |-- new/page.tsx          # Start new mock interview
| | | | |-- [id]/page.tsx         # Interview session
| | | | |-- [id]/replay/page.tsx  # Interview replay
| | | |-- resume/page.tsx         # Resume upload + insights
| | | |-- recommendations/page.tsx # Learning recommendations
| | | |-- readiness/page.tsx      # Placement readiness
| | | |-- search/page.tsx         # Knowledge base search
| | | |-- weaknesses/page.tsx     # Weakness analysis
| | |-- (admin)/
| | | |-- layout.tsx              # Admin layout
| | | |-- admin/page.tsx          # Admin dashboard
| | | |-- admin/companies/page.tsx
| | | |-- admin/experiences/page.tsx
| | | |-- admin/analytics/page.tsx
| |-- components/
| | |-- ui/                       # Shadcn UI components
| | |-- layout/
| | | |-- Sidebar.tsx
| | | |-- Header.tsx
| | | |-- Footer.tsx
| | |-- dashboard/
| | | |-- ProfileSummary.tsx
| | | |-- PlacementStats.tsx
| | | |-- RecentActivity.tsx
| | | |-- PerformanceOverview.tsx
| | | |-- TrendGraph.tsx
| | |-- experiences/
| | | |-- ExperienceForm.tsx
| | | |-- ExperienceCard.tsx
| | | |-- QuestionEntry.tsx
| | |-- companies/
| | | |-- CompanyCard.tsx
| | | |-- CompanyIntelligence.tsx
| | | |-- TopicFrequency.tsx
| | | |-- DifficultyTrends.tsx
| | |-- interviews/
| | | |-- InterviewChat.tsx
| | | |-- InterviewConfig.tsx
| | | |-- VoiceRecorder.tsx
| | | |-- ReplayViewer.tsx
| | | |-- EvaluationDisplay.tsx
| | |-- resume/
| | | |-- ResumeUpload.tsx
| | | |-- ResumeInsights.tsx
| |-- readiness/

```

```
| | | |-- ReadinessScore.tsx
| | | |-- ReadinessBreakdown.tsx
| | |-- recommendations/
| | | |-- TopicCard.tsx
| | | |-- PracticePlan.tsx
| | |-- search/
| | | |-- SearchBar.tsx
| | | |-- SearchResults.tsx
| |-- lib/
| | |-- api.ts                # Axios/fetch wrapper
| | |-- auth.ts              # Supabase auth helpers
| | |-- supabase.ts          # Supabase client
| | |-- utils.ts             # Utility functions
| | |-- constants.ts         # App constants
| |-- hooks/
| | |-- useAuth.ts
| | |-- useDashboard.ts
| | |-- useInterview.ts
| | |-- useCompany.ts
| | |-- useVoice.ts
| |-- stores/
| | |-- authStore.ts         # Zustand auth state
| | |-- interviewStore.ts    # Zustand interview state
| |-- types/
| | |-- index.ts             # All TypeScript interfaces
|-- public/
|-- tailwind.config.ts
|-- next.config.ts
|-- tsconfig.json
|-- package.json
```

Section 5: Database Schema (All Tables)

Normalized PostgreSQL schema with 15+ tables. All tables use UUID primary keys. All timestamps are timezone-aware. Designed for multi-tenancy and future Mentor role expansion.

5.1 Table: roles

```
CREATE TABLE roles (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name        VARCHAR(50) UNIQUE NOT NULL,      -- student, admin, mentor  
  description TEXT,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

5.2 Table: permissions

```
CREATE TABLE permissions (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  role_id     UUID REFERENCES roles(id),  
  resource     VARCHAR(100) NOT NULL,            -- companies, experiences, etc.  
  action       VARCHAR(50) NOT NULL,            -- create, read, update, delete  
  UNIQUE(role_id, resource, action)  
);
```

5.3 Table: users

```
CREATE TABLE users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  hashed_password VARCHAR(255),                -- NULL for OAuth users  
  auth_provider VARCHAR(50) DEFAULT 'email', -- email, google  
  supabase_uid VARCHAR(255) UNIQUE,  
  is_active   BOOLEAN DEFAULT TRUE,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

5.4 Table: user_roles

```
CREATE TABLE user_roles (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,  
  role_id     UUID REFERENCES roles(id),  
  UNIQUE(user_id, role_id)  
);
```

5.5 Table: profiles

```
CREATE TABLE profiles (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,  
  full_name   VARCHAR(255) NOT NULL,  
  kalvium_id  VARCHAR(100) UNIQUE,
```

```
batch          VARCHAR(50),
graduation_year INTEGER,
resume_url     TEXT,
linkedin_url   TEXT,
github_url     TEXT,
target_companies JSONB DEFAULT '[]',
created_at     TIMESTAMPTZ DEFAULT NOW(),
updated_at     TIMESTAMPTZ DEFAULT NOW()
);
```

5.6 Table: companies

```
CREATE TABLE companies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(255) UNIQUE NOT NULL,
  industry    VARCHAR(100),
  website     TEXT,
  logo_url    TEXT,
  description TEXT,
  is_active   BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);
```

5.7 Table: interview_experiences

```
CREATE TABLE interview_experiences (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID REFERENCES users(id) ON DELETE CASCADE,
  company_id        UUID REFERENCES companies(id),
  role              VARCHAR(255) NOT NULL,
  interview_date    DATE NOT NULL,
  round_type        VARCHAR(50) NOT NULL,      -- OA, Technical, HM, HR, System Design, Other
  difficulty        VARCHAR(20) NOT NULL,      -- Easy, Medium, Hard
  outcome           VARCHAR(20) NOT NULL,      -- Selected, Rejected, Waiting, Unknown
  student_notes     TEXT,
  ai_extracted      JSONB,                    -- AI-generated structured metadata
  is_approved       BOOLEAN DEFAULT FALSE,
  is_flagged        BOOLEAN DEFAULT FALSE,
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
);
```

5.8 Table: interview_questions

```
CREATE TABLE interview_questions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  experience_id      UUID REFERENCES interview_experiences(id) ON DELETE CASCADE,
  topic             VARCHAR(100) NOT NULL,      -- Arrays, Binary Search, SQL, React, etc.
  question_text     TEXT NOT NULL,
  could_answer      VARCHAR(20) NOT NULL,      -- Yes, Partially, No
  ai_tags           JSONB,                    -- AI-extracted tags
  embedding          vector(768),             -- pgvector for semantic search
  created_at        TIMESTAMPTZ DEFAULT NOW()
);
```

5.9 Table: interview_outcomes (analytics helper)

```
CREATE TABLE interview_outcomes (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  experience_id      UUID UNIQUE REFERENCES interview_experiences(id) ON DELETE CASCADE,
  company_id        UUID REFERENCES companies(id),
  success           BOOLEAN,
  topics            JSONB,                    -- aggregated topics
  skills_tested     JSONB,
  created_at        TIMESTAMPTZ DEFAULT NOW()
);
```

5.10 Table: company_analytics

```
CREATE TABLE company_analytics (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id        UUID UNIQUE REFERENCES companies(id),
  total_experiences INTEGER DEFAULT 0,
  total_questions  INTEGER DEFAULT 0,
  top_topics        JSONB DEFAULT '[]',
  difficulty_dist    JSONB DEFAULT '{}',      -- {easy: N, medium: N, hard: N}
  round_dist        JSONB DEFAULT '{}',
  success_rate       FLOAT DEFAULT 0,
);
```

```
common_weaknesses    JSONB DEFAULT '[]',
recent_90d_weight     FLOAT DEFAULT 1.0,
last_computed         TIMESTAMPTZ DEFAULT NOW(),
updated_at            TIMESTAMPTZ DEFAULT NOW()
);
```

5.11 Table: mock_interviews

```
CREATE TABLE mock_interviews (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID REFERENCES users(id) ON DELETE CASCADE,
  company_id        UUID REFERENCES companies(id),           -- NULL for generic
  interview_type     VARCHAR(50) NOT NULL,                   -- behavioral, technical, hm, project, company,
  custom
  difficulty         VARCHAR(20) NOT NULL,
  mode               VARCHAR(20) DEFAULT 'text', -- text, voice
  status             VARCHAR(20) DEFAULT 'in_progress', -- in_progress, completed, abandoned
  context            JSONB,                                  -- resume data + company data used
  started_at        TIMESTAMPTZ DEFAULT NOW(),
  completed_at       TIMESTAMPTZ,
  created_at         TIMESTAMPTZ DEFAULT NOW()
);
```

5.12 Table: mock_interview_questions

```
CREATE TABLE mock_interview_questions (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  interview_id      UUID REFERENCES mock_interviews(id) ON DELETE CASCADE,
  sequence_num      INTEGER NOT NULL,
  question_text     TEXT NOT NULL,
  question_type     VARCHAR(50),                -- main, follow_up
  topic             VARCHAR(100),
  student_answer    TEXT,
  ideal_answer      TEXT,                      -- AI-generated ideal answer
  audio_url         TEXT,                      -- voice mode audio storage
  evaluation        JSONB,                     -- per-question evaluation
  feedback          TEXT,                      -- per-question feedback
  created_at        TIMESTAMPTZ DEFAULT NOW()
);
```

5.13 Table: evaluations

```
CREATE TABLE evaluations (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  interview_id      UUID UNIQUE REFERENCES mock_interviews(id) ON DELETE CASCADE,
  communication_score FLOAT NOT NULL,
  communication_reason TEXT NOT NULL,
  technical_score   FLOAT NOT NULL,
  technical_reason  TEXT NOT NULL,
  confidence_score  FLOAT NOT NULL,
  confidence_reason TEXT NOT NULL,
  problem_solving_score FLOAT NOT NULL,
  problem_solving_reason TEXT NOT NULL,
  behavioral_score  FLOAT NOT NULL,
  behavioral_reason TEXT NOT NULL,
  project_score     FLOAT NOT NULL,
  project_reason    TEXT NOT NULL,
  overall_score     FLOAT NOT NULL,
  overall_feedback  TEXT NOT NULL,
  strengths         JSONB DEFAULT '[]',
  improvements      JSONB DEFAULT '[]',
  created_at        TIMESTAMPTZ DEFAULT NOW()
);
```

5.14 Table: weaknesses

```
CREATE TABLE weaknesses (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID REFERENCES users(id) ON DELETE CASCADE,
  topic             VARCHAR(100) NOT NULL,
  category          VARCHAR(50),                -- technical, behavioral, communication
  occurrence_count  INTEGER DEFAULT 1,
  severity          VARCHAR(20),                -- low, medium, high
  sources           JSONB DEFAULT '[]',         -- list of interview IDs where detected
  recommended_actions JSONB DEFAULT '[]',
  is_resolved       BOOLEAN DEFAULT FALSE,
  first_detected    TIMESTAMPTZ DEFAULT NOW(),
  last_detected     TIMESTAMPTZ DEFAULT NOW()
);
```



```
);
```

5.15 Table: recommendations

```
CREATE TABLE recommendations (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           UUID REFERENCES users(id) ON DELETE CASCADE,  
  type              VARCHAR(50) NOT NULL,          -- topic, practice_plan, mock_interview  
  title             VARCHAR(255) NOT NULL,  
  description        TEXT,  
  priority           INTEGER DEFAULT 0,            -- higher = more important  
  metadata           JSONB DEFAULT '{}',  
  is_completed       BOOLEAN DEFAULT FALSE,  
  created_at         TIMESTAMPTZ DEFAULT NOW()  
);
```

5.16 Table: resume_data

```
CREATE TABLE resume_data (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,  
  raw_text           TEXT,  
  skills             JSONB DEFAULT '[]',  
  projects           JSONB DEFAULT '[]',  
  experience          JSONB DEFAULT '[]',  
  technologies        JSONB DEFAULT '[]',  
  domains            JSONB DEFAULT '[]',  
  insights           JSONB DEFAULT '{}',          -- missing skills, strengths, etc.  
  embedding          vector(768),  
  parsed_at          TIMESTAMPTZ DEFAULT NOW(),  
  updated_at         TIMESTAMPTZ DEFAULT NOW()  
);
```

5.17 Table: activity_logs

```
CREATE TABLE activity_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  action      VARCHAR(100) NOT NULL,
  resource    VARCHAR(100),
  resource_id UUID,
  metadata    JSONB DEFAULT '{}',
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_activity_logs_user ON activity_logs(user_id);
CREATE INDEX idx_activity_logs_created ON activity_logs(created_at DESC);
CREATE INDEX idx_interview_questions_embedding ON interview_questions
  USING ivfflat (embedding vector_cosine_ops);
CREATE INDEX idx_interview_experiences_company ON interview_experiences(company_id);
CREATE INDEX idx_interview_experiences_date ON interview_experiences(interview_date DESC);
CREATE INDEX idx_mock_interviews_user ON mock_interviews(user_id);
CREATE INDEX idx_weaknesses_user ON weaknesses(user_id);
```

5.18 Entity Relationship Summary

```
User 1--1 Profile
User 1--* UserRole --* Role 1--* Permission
User 1--* InterviewExperience *--1 Company
InterviewExperience 1--* InterviewQuestion
InterviewExperience 1--1 InterviewOutcome
Company 1--1 CompanyAnalytics
User 1--* MockInterview *--1 Company (optional)
MockInterview 1--* MockInterviewQuestion
MockInterview 1--1 Evaluation
User 1--* Weakness
User 1--* Recommendation
User 1--1 ResumeData
User 1--* ActivityLog
```

Section 6: Module 1: Authentication System

Email/password + Google OAuth via Supabase Auth. JWT-based session management. Role-based access control.

API Endpoints

Method	Path	Description	Auth
POST	/api/v1/auth/register	Register with email/password	Public
POST	/api/v1/auth/login	Login with email/password	Public
POST	/api/v1/auth/google	Google OAuth callback	Public
POST	/api/v1/auth/refresh	Refresh JWT token	Authenticated
POST	/api/v1/auth/logout	Invalidate session	Authenticated

Frontend Pages

- > (auth)/login/page.tsx
- > (auth)/register/page.tsx

Components

- > LoginForm
- > RegisterForm
- > GoogleAuthButton
- > AuthGuard (HOC)

Backend Services

- > auth_service.py: register_user(), login_user(), google_auth(), refresh_token()

Implementation Logic (Step-by-Step)

- > 1. On register: create User + Profile + assign 'student' role
- > 2. On Google OAuth: exchange code with Supabase, create/link user
- > 3. JWT contains: user_id, email, role, exp
- > 4. Frontend stores JWT in httpOnly cookie (secure) or localStorage
- > 5. Every protected API route uses get_current_user dependency
- > 6. Admin routes use require_admin dependency

Section 7: Module 2: Student Dashboard

Aggregated view of student's profile, placement stats, recent activity, and performance overview with trend graphs.

API Endpoints

Method	Path	Description	Auth
GET	/api/v1/dashboard	Get full dashboard data	Student
GET	/api/v1/users/me	Get current user info	Authenticated
PUT	/api/v1/users/me/profile	Update profile	Student

Frontend Pages

- > (dashboard)/dashboard/page.tsx
- > (dashboard)/profile/page.tsx

Components

- > ProfileSummary
- > PlacementStats
- > RecentActivity
- > PerformanceOverview
- > TrendGraph (Recharts)

Backend Services

- > dashboard_service.py: get_dashboard_data(), get_performance_trends()

Implementation Logic (Step-by-Step)

- > 1. Profile Summary: name, batch, resume status, target companies
- > 2. Placement Stats: interviews attempted, AI interviews completed, contributions, reports viewed
- > 3. Recent Activity: last 10 actions from activity_logs
- > 4. Performance Overview: avg scores across Communication, Technical, Problem Solving, Behavioral, Project
- > 5. Trend Graphs: scores over time using Recharts line charts (last 10 interviews)
- > 6. All stats computed via SQL aggregation queries, NOT stored as denormalized data

Section 8: Module 3: Interview Experience Collection

The MOST important module. Creates a continuously growing interview intelligence database.

API Endpoints

Method	Path	Description	Auth
POST	/api/v1/experiences	Submit new experience	Student
GET	/api/v1/experiences	List experiences (paginated)	Student
GET	/api/v1/experiences/{id}	Get experience detail	Student
PUT	/api/v1/experiences/{id}	Update experience	Owner/Admin
DELETE	/api/v1/experiences/{id}	Delete experience	Owner/Admin

Frontend Pages

- > experiences/page.tsx
- > experiences/new/page.tsx
- > experiences/[id]/page.tsx

Components

- > ExperienceForm
- > QuestionEntry (repeatable)
- > ExperienceCard
- > ExperienceDetail

Backend Services

- > experience_service.py: create_experience(), ai/knowledge_extractor.py: extract_metadata()

Implementation Logic (Step-by-Step)

- > 1. Form fields: Company, Role, Date, Round Type (OA/Technical/HM/HR/System Design/Other)
- > 2. Difficulty (Easy/Medium/Hard), Outcome (Selected/Rejected/Waiting/Unknown)
- > 3. Questions: repeatable section - each has Topic, Question Text, Could Answer (Yes/Partially/No)
- > 4. Student Notes: optional free text
- > 5. ON SUBMIT: Save raw data immediately, then trigger async AI processing via Celery
- > 6. AI Processing: Extract topics, skills, technologies, patterns -> store in ai_extracted JSONB
- > 7. AI also generates embedding for each question -> store in interview_questions.embedding
- > 8. After AI processing, update company_analytics (recompute aggregations)
- > 9. Admin can moderate: approve, flag, reject submissions

Section 9: Module 4: Company Intelligence Engine

Transforms raw interview experiences into actionable company-specific insights.

API Endpoints

Method	Path	Description	Auth
GET	/api/v1/companies	List companies with stats	Student
GET	/api/v1/companies/{id}	Full company intelligence	Student
POST	/api/v1/companies	Create company	Admin
PUT	/api/v1/companies/{id}	Update company	Admin

Frontend Pages

- > companies/page.tsx
- > companies/[id]/page.tsx

Components

- > CompanyCard
- > CompanyIntelligence
- > TopicFrequency
- > DifficultyTrends
- > RoundBreakdown
- > SuccessPatterns
- > RecentExperiences

Backend Services

- > company_service.py: get_company_intelligence(), recompute_analytics()

Implementation Logic (Step-by-Step)

- > 1. Company Page displays: Total Experiences, Interview Rounds breakdown, Most Asked Topics
- > 2. Frequently Asked Questions (ranked by frequency), Difficulty Trends over time
- > 3. Recent Experiences (chronological), Success Patterns, Common Weaknesses
- > 4. Questions grouped by Round (Technical, HM, HR, OA)
- > 5. TIME SENSITIVITY: Recent data (last 90 days) gets higher weight
- > 6. Weighting formula: $\text{weight} = 1.0 \text{ if within } 90 \text{ days, else } 0.5 * (1 - \text{days_old}/365)$
- > 7. company_analytics table is recomputed after each new experience submission
- > 8. Analytics recomputation runs as async Celery task

Section 10: Module 5: Resume Intelligence

AI-powered resume parsing, skill extraction, and gap analysis.

API Endpoints

Method	Path	Description	Auth
POST	/api/v1/resume/upload	Upload resume (PDF/text)	Student
GET	/api/v1/resume/insights	Get resume insights	Student
GET	/api/v1/resume/data	Get parsed resume data	Student

Frontend Pages

> resume/page.tsx

Components

- > ResumeUpload (drag-drop)
- > ResumeInsights
- > SkillsGrid
- > MissingSkillsAlert

Backend Services

> ai/resume_parser.py: parse_resume(), generate_insights()

Implementation Logic (Step-by-Step)

- > 1. Accept PDF or plain text resume upload
- > 2. Store file in Supabase Storage, save URL in profile.resume_url
- > 3. AI extracts: Skills, Projects, Experience, Technologies, Domains
- > 4. Store structured data in resume_data table
- > 5. Generate insights: Missing Skills (vs target companies), Strength Areas, Potential Interview Topics
- > 6. Generate embedding from resume text -> store in resume_data.embedding
- > 7. Resume data is used by Mock Interview Generator for personalized questions
- > 8. Re-parsing on re-upload replaces old data

Section 11: Module 6: AI Mock Interview System

The core AI engine. Resume-aware, company-specific, adaptive mock interviews.

API Endpoints

Method	Path	Description	Auth
POST	/api/v1/interviews/start	Start new mock interview	Student
POST	/api/v1/interviews/{id}/answer	Submit answer + get next Q	Student
POST	/api/v1/interviews/{id}/complete	End interview + get eval	Student
GET	/api/v1/interviews	List past interviews	Student
GET	/api/v1/interviews/{id}	Get interview detail	Student

Frontend Pages

- > interviews/page.tsx
- > interviews/new/page.tsx
- > interviews/[id]/page.tsx

Components

- > InterviewConfig
- > InterviewChat
- > AnswerInput
- > VoiceRecorder
- > InterviewTimer

Backend Services

- > mock_interview_service.py, ai/interview_generator.py, ai/followup_generator.py

Implementation Logic (Step-by-Step)

- > 1. Interview Types: Behavioral, Technical, Hiring Manager, Project Discussion, Company Specific, Custom
- > 2. Difficulty Modes: Easy, Medium, Hard
- > 3. Modes: Text, Voice
- > 4. START: Student picks type + difficulty + company (optional) + mode
- > 5. System loads: student's resume_data + target company intelligence + past interview history
- > 6. Question Generation prompt includes ALL context: resume skills, company topics, past weaknesses
- > 7. RESUME-AWARE: If resume mentions FastAPI, ask 'Why FastAPI?', 'async in FastAPI?', 'FastAPI vs Flask?'
- > 8. ADAPTIVE FOLLOW-UPS: After each answer, AI generates follow-up based on answer quality
- > 9. Do NOT use fixed question flows. Every interview is dynamic.
- > 10. Each Q&A is stored in mock_interview_questions with sequence_num
- > 11. On COMPLETE: trigger full evaluation via EvaluatorService

- > 12. Interview state managed on backend (stateless API, state in DB)
- > 13. Typical interview: 8-12 questions including follow-ups

Section 12: Module 7: Voice Interview Engine

Full voice pipeline: speech-to-text, processing, text-to-speech.

API Endpoints

Method	Path	Description	Auth
POST	/api/v1/voice/transcribe	Convert speech to text	Student
POST	/api/v1/voice/synthesize	Convert text to speech	Student

Frontend Pages

- > (reuses interview/[id]/page.tsx with voice mode)

Components

- > VoiceRecorder (MediaRecorder API)
- > AudioPlayer
- > VoiceStatus indicator

Backend Services

- > ai/voice_engine.py: transcribe(), synthesize()

Implementation Logic (Step-by-Step)

- > 1. Voice Input: Browser MediaRecorder API captures audio -> send as blob to /voice/transcribe
- > 2. Whisper (openai-whisper) transcribes audio to text on backend
- > 3. Transcribed text feeds into normal interview processing pipeline
- > 4. AI response text is synthesized to speech via Piper TTS
- > 5. Audio response sent back to frontend as base64 or streaming audio
- > 6. Store audio URLs in mock_interview_questions.audio_url
- > 7. Fallback: if voice fails, gracefully degrade to text mode
- > 8. Latency target: < 3 seconds for transcribe, < 2 seconds for synthesize

Section 13: Module 8: Evaluation Engine

Explainable scoring across 6 dimensions. NO arbitrary scores.

API Endpoints

Method	Path	Description	Auth
GET	/api/v1/evaluations/{interview_id}	Get evaluation for interview	Student

Frontend Pages

> (part of interviews/[id]/page.tsx after completion)

Components

> EvaluationDisplay
> ScoreCard
> RadarChart
> FeedbackSection

Backend Services

> ai/evaluator.py: evaluate_interview()

Implementation Logic (Step-by-Step)

> 1. 6 Scoring Dimensions: Communication, Technical Depth, Confidence, Problem Solving, Behavioral, Project Understanding
> 2. Each dimension: score (1-10) + reason (text explanation)
> 3. Example: Communication: 7/10, Reason: 'Clear explanations but frequent pauses'
> 4. Overall score = weighted average of all dimensions
> 5. Also generates: strengths list, improvements list, overall feedback paragraph
> 6. Evaluator receives full interview transcript (all Q&A pairs)
> 7. AI prompt enforces: 'Every score must have a specific reason citing student answers'
> 8. Scores stored in evaluations table
> 9. Per-question evaluation stored in mock_interview_questions.evaluation JSONB

Section 14: Module 9: Interview Replay

Students can review past interviews with Q&A, evaluations, and ideal answers.

API Endpoints

Method	Path	Description	Auth
GET	/api/v1/interviews/{id}/replay	Get full replay data	Student

Frontend Pages

> interviews/[id]/replay/page.tsx

Components

- > ReplayViewer
- > QAPair
- > IdealAnswerCompare
- > ReplayEvaluation

Backend Services

> mock_interview_service.py: get_replay_data()

Implementation Logic (Step-by-Step)

- > 1. Display chronological list of: Question -> Student Answer -> Evaluation -> Feedback -> Ideal Answer
- > 2. For voice interviews: include audio playback for each Q&A
- > 3. Show per-question scores alongside overall evaluation
- > 4. Highlight areas where student answer diverged from ideal answer
- > 5. Include overall evaluation summary at the end
- > 6. Allow navigation between questions

Section 15: Module 10: Weakness Detection Engine

Tracks recurring weaknesses across multiple interviews and suggests targeted actions.

API Endpoints

Method	Path	Description	Auth
GET	/api/v1/weaknesses	Get user's weaknesses	Student
PUT	/api/v1/weaknesses/{id}/resolve	Mark weakness resolved	Student

Frontend Pages

- > weaknesses/page.tsx

Components

- > WeaknessCard
- > WeaknessTimeline
- > ActionItems

Backend Services

- > ai/evaluator.py: detect_weaknesses() (runs after each evaluation)

Implementation Logic (Step-by-Step)

- > 1. After every mock interview evaluation, scan for weak topics
- > 2. If a topic scores below threshold (< 5/10) across 2+ interviews -> create/update Weakness record
- > 3. Example: SQL appears weak in Mock 1, 2, 3 -> Recurring Weakness: SQL Joins
- > 4. Severity: low (2 occurrences), medium (3-4), high (5+)
- > 5. Each weakness has recommended_actions: 'Practice SQL Sheet', 'Take SQL Mock'
- > 6. Track sources (which interviews detected this weakness)
- > 7. Student can mark weakness as 'resolved' after targeted practice
- > 8. Weakness data feeds into Recommendation Engine and Readiness Engine

Section 16: Module 11: Learning Recommendations

Personalized study plans based on resume, history, weaknesses, and target companies.

API Endpoints

Method	Path	Description	Auth
GET	/api/v1/recommendations	Get personalized recommendations	Student
PUT	/api/v1/recommendations/{id}/complete	Mark recommendation done	Student

Frontend Pages

> recommendations/page.tsx

Components

> TopicCard
> PracticePlan
> SuggestedMocks
> PriorityList

Backend Services

> recommendation_service.py, ai/recommendation_engine.py

Implementation Logic (Step-by-Step)

- > 1. Inputs: Resume data + Interview history + Weaknesses + Target companies
- > 2. Generates: Topics to Study (ranked by priority), Practice Plans, Suggested Mock Interviews
- > 3. Priority ranking based on: weakness severity + target company requirements + recency
- > 4. Example: If targeting Lowe's and weak in SQL -> SQL is highest priority
- > 5. Recommendations are regenerated after each mock interview or experience submission
- > 6. Student can mark recommendations as completed
- > 7. Completed recommendations are excluded from future generation

Section 17: Module 12: Placement Readiness Engine

Explainable placement readiness score per company.

API Endpoints

Method	Path	Description	Auth
GET	/api/v1/readiness	Overall readiness	Student
GET	/api/v1/readiness/{company_id}	Company-specific readiness	Student

Frontend Pages

- > readiness/page.tsx

Components

- > ReadinessScore (circular progress)
- > ReadinessBreakdown
- > CompanyReadinessCard
- > GapAnalysis

Backend Services

- > readiness_service.py: compute_readiness()

Implementation Logic (Step-by-Step)

- > 1. Computes readiness percentage for each target company
- > 2. Example: Lowe's Readiness: 78%
- > 3. Based on: Technical (vs company requirements), Communication, Behavioral, Project, Company-specific competencies
- > 4. Each dimension shows score + gap explanation
- > 5. Uses: company_analytics (what company asks) + evaluations (student performance) + weaknesses
- > 6. NEVER generate opaque scores. Every percentage must be traceable.
- > 7. Formula: $\text{readiness} = \text{weighted_avg}(\text{dimension_scores})$ where weights come from company topic distribution
- > 8. Display recommendations to close gaps

Section 18: Module 13: Knowledge Base Search

Search across all interview experiences by company, topic, question, technology.

API Endpoints

Method	Path	Description	Auth
GET	/api/v1/search?q=...&type=...&company=...	Search knowledge base	Student

Frontend Pages

> search/page.tsx

Components

- > SearchBar (with filters)
- > SearchResults
- > QuestionResult
- > CompanyResult

Backend Services

> knowledge_base_service.py: search()

Implementation Logic (Step-by-Step)

- > 1. Search types: company, topic, question (text), technology
- > 2. Text search: PostgreSQL full-text search on interview_questions.question_text
- > 3. Semantic search: pgvector cosine similarity on question embeddings
- > 4. Hybrid: combine text + semantic results with RRF (Reciprocal Rank Fusion)
- > 5. Example: Search 'Binary Search' -> Companies that asked it, Frequency, Difficulty, Related Questions
- > 6. Results show: company name, round type, difficulty, question text, frequency count
- > 7. Filter by: company, round type, difficulty, date range
- > 8. Pagination with cursor-based approach for performance

Section 19: Complete API Contract

Base URL: /api/v1 | All responses follow: { success: bool, data: T, error: string | null }

Auth

Method	Path	Request	Response
POST	/auth/register	{email, password, full_name}	{user, token}
POST	/auth/login	{email, password}	{user, token}
POST	/auth/google	{access_token}	{user, token}
POST	/auth/refresh	Header: Bearer token	{token}

User

Method	Path	Request	Response
GET	/users/me	-	{user, profile}
PUT	/users/me/profile	{full_name, batch, ...}	{profile}

Dashboard

Method	Path	Request	Response
GET	/dashboard	-	{stats, recent, performance, trends}

Experiences

Method	Path	Request	Response
POST	/experiences	{company_id, role, ...questions}	{experience}
GET	/experiences?page&limit	-	{experiences[], total}
GET	/experiences/:id	-	{experience, questions}

Companies

Method	Path	Request	Response
GET	/companies?search	-	{companies[]}
GET	/companies/:id	-	{company, analytics, questions}

Interviews

Method	Path	Request	Response
POST	/interviews/start	{type, difficulty, company_id, mode}	{interview, first_question}
POST	/interviews/:id/answer	{answer, audio_blob?}	{next_question, evaluation?}
POST	/interviews/:id/complete	-	{evaluation}
GET	/interviews	-	{interviews[]}
GET	/interviews/:id/replay	-	{questions[], evaluation}

Resume

Method	Path	Request	Response
POST	/resume/upload	multipart/form-data	{resume_data}
GET	/resume/insights	-	{insights}

Intelligence

Method	Path	Request	Response
GET	/weaknesses	-	{weaknesses[]}
GET	/recommendations	-	{recommendations[]}
GET	/readiness	-	{overall, companies[]}
GET	/readiness/:company_id	-	{readiness, breakdown}
GET	/search?q&type&company	-	{results[]}

Voice

Method	Path	Request	Response
POST	/voice/transcribe	audio blob	{text}
POST	/voice/synthesize	{text}	{audio_base64}

Admin

Method	Path	Request	Response
GET	/admin/experiences?status	-	{experiences[]}
PUT	/admin/experiences/:id/approve	-	{experience}
PUT	/admin/experiences/:id/flag	{reason}	{experience}
GET	/admin/analytics	-	{platform_stats}

Section 20: Frontend Pages & Component Tree

20.1 Page Route Map

Route	Description
/	Landing page - hero + features + CTA
/login	Email/password + Google OAuth login
/register	Registration form
/dashboard	Student dashboard - stats, trends, activity
/profile	Profile management + target companies
/experiences	List all submitted experiences
/experiences/new	Submit new interview experience
/experiences/[id]	Experience detail view
/companies	Company directory with search
/companies/[id]	Company intelligence page
/interviews	Mock interview history
/interviews/new	Configure + start new interview
/interviews/[id]	Active interview session (chat/voice)
/interviews/[id]/replay	Interview replay view
/resume	Resume upload + insights
/weaknesses	Weakness analysis dashboard
/recommendations	Personalized recommendations
/readiness	Placement readiness scores
/search	Knowledge base search
/admin	Admin dashboard
/admin/companies	Manage companies
/admin/experiences	Moderate experiences
/admin/analytics	Platform analytics

20.2 Component Architecture

Layout Components:

- RootLayout (app/layout.tsx): Providers, fonts, global CSS
- AuthLayout ((auth)/layout.tsx): Centered card layout
- DashboardLayout ((dashboard)/layout.tsx): Sidebar + Header + Main
- AdminLayout ((admin)/layout.tsx): Admin sidebar + header

Shared Components (components/ui/):

- All Shadcn UI components: Button, Card, Input, Select, Dialog, etc.
- Custom: LoadingSpinner, EmptyState, ErrorBoundary, Pagination

State Management:

- Zustand stores: authStore (user, token), interviewStore (session state)
- TanStack Query: all server data fetching with caching
- React Hook Form + Zod: all form handling

Data Flow:

- Page -> useQuery(hook) -> API call -> Backend -> DB
- Page -> Form submit -> useMutation -> API call -> Invalidate queries

Section 21: AI Service Architecture

21.1 Base Interface

```
class BaseAIService(ABC):
    @abstractmethod
    async def generate(self, prompt: str, context: dict) -> dict:
        pass

class GeminiClient(BaseAIService):
    def __init__(self, api_key: str, model: str = 'gemini-2.0-flash'):
        self.client = genai.configure(api_key=api_key)
        self.model = genai.GenerativeModel(model)

    async def generate(self, prompt: str, context: dict) -> dict:
        response = await self.model.generate_content_async(prompt)
        return parse_json(response.text)
```

21.2 Service Details

ResumeParser

Input: PDF/text resume
Output: {skills[], projects[], experience[], technologies[], domains[]}
Prompt: Extract structured data from resume, identify key skills and technologies.
Also generates: insights (missing skills vs market, strengths, interview topic predictions)

InterviewGenerator

Input: {resume_data, company_analytics, interview_type, difficulty, past_weaknesses}
Output: {question, topic, expected_depth}
Prompt includes full context. Generates resume-aware, company-specific questions.
Example: Resume has FastAPI -> 'Explain async in FastAPI', 'FastAPI vs Flask?'

FollowUpGenerator

Input: {question, student_answer, answer_quality, interview_context}
Output: {follow_up_question, reasoning}
Rules: MUST be dynamic. NEVER use fixed flows.
If answer is shallow -> dig deeper. If answer is wrong -> redirect. If good -> advance topic.

Evaluator

Input: {full_transcript: [{question, answer}], interview_type}
Output: {scores: {communication, technical, confidence, problem_solving, behavioral, project}, reasons, strengths, improvements}
Rules: EVERY score must cite specific answer evidence. No arbitrary scores.

RecommendationEngine

Input: {resume, weaknesses, evaluations, target_companies}
Output: {topics_to_study[], practice_plans[], suggestedmocks[]}
Priority based on: weakness severity x company relevance x recency

KnowledgeExtractor

```
Input: {raw_experience, questions[]}
```

```
Output: {topics[], skills[], technologies[], patterns[], tags[]}
```

```
Runs async after experience submission. Enriches raw data with structured metadata.
```

21.3 Prompt Engineering Guidelines

- > All prompts return structured JSON (not free text)
- > Use system prompts to set role and constraints
- > Include few-shot examples in prompts for consistent output
- > Temperature: 0.3 for evaluation (consistency), 0.7 for question generation (variety)
- > Max tokens: calibrated per service (eval ~2000, questions ~500)
- > Error handling: retry 3x with exponential backoff, fallback to simpler prompt
- > All AI responses are validated against Pydantic schemas before storage

Section 22: Environment & Deployment Guide

22.1 Local Development Setup

```
# 1. Clone repo
git clone <repo-url>
cd placement-intelligence-platform

# 2. Backend setup
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # Fill in values
alembic upgrade head
uvicorn app.main:app --reload --port 8000

# 3. Frontend setup
cd frontend
npm install
cp .env.example .env.local # Fill in values
npm run dev

# 4. Redis (for Celery)
brew install redis && redis-server

# 5. Celery worker
cd backend
celery -A app.worker worker --loglevel=info
```

22.2 Environment Variables

Variable	Description	Where
DATABASE_URL	PostgreSQL connection string	Backend .env
SUPABASE_URL	Supabase project URL	Both
SUPABASE_KEY	Supabase anon key	Both
SUPABASE_SERVICE_KEY	Supabase service role key	Backend .env
GEMINI_API_KEY	Google Gemini API key	Backend .env
JWT_SECRET	JWT signing secret	Backend .env
REDIS_URL	Redis connection URL	Backend .env
NEXT_PUBLIC_API_URL	Backend API base URL	Frontend .env
NEXT_PUBLIC_SUPABASE_URL	Supabase URL	Frontend .env
NEXT_PUBLIC_SUPABASE_KEY	Supabase anon key	Frontend .env

22.3 Production Deployment

Frontend (Vercel):

- Connect GitHub repo

- Set root directory: frontend/
- Framework: Next.js
- Set environment variables in Vercel dashboard
- Auto-deploys on push to main

Backend (Render):

- Connect GitHub repo
- Service type: Web Service
- Root directory: backend/
- Build command: `pip install -r requirements.txt`
- Start command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- Set environment variables
- Add Redis addon for Celery

Database (Supabase):

- Create project at supabase.com
- Enable pgvector extension: `CREATE EXTENSION vector;`
- Run migrations via alembic or Supabase SQL editor
- Enable Auth providers (Email + Google)

Section 23: Build Order & Dependency Graph

Modules MUST be built in this order due to dependencies. Each phase builds on the previous. Do NOT skip ahead.

Phase 1: Foundation

- [] 1.1 Project scaffolding (dirs, configs, package.json, requirements.txt)
- [] 1.2 Database models (all 17 tables in SQLAlchemy)
- [] 1.3 Alembic migrations setup + initial migration
- [] 1.4 FastAPI app entry point (main.py) with CORS, routers
- [] 1.5 Pydantic schemas (all request/response schemas)
- [] 1.6 Frontend: Next.js init + Tailwind + Shadcn setup
- [] 1.7 Frontend: Layout components (RootLayout, Sidebar, Header)
- [] 1.8 Docker compose for local Postgres + Redis

Phase 2: Auth + User

- [] 2.1 Backend: auth_service.py (register, login, Google OAuth)
- [] 2.2 Backend: auth routes (register, login, google, refresh)
- [] 2.3 Backend: security middleware (JWT validation, RBAC)
- [] 2.4 Backend: user routes (get profile, update profile)
- [] 2.5 Frontend: Supabase client setup
- [] 2.6 Frontend: Login page + Register page
- [] 2.7 Frontend: Auth store (Zustand) + AuthGuard HOC
- [] 2.8 Frontend: Profile page

Phase 3: Dashboard

- [] 3.1 Backend: dashboard_service.py (aggregate stats, trends)
- [] 3.2 Backend: dashboard route
- [] 3.3 Frontend: Dashboard page with all 4 sections
- [] 3.4 Frontend: TrendGraph component (Recharts)

Phase 4: Interview Experiences

- [] 4.1 Backend: experience_service.py (CRUD)
- [] 4.2 Backend: experience routes
- [] 4.3 Backend: AI knowledge_extractor.py (async processing)
- [] 4.4 Frontend: Experience submission form (multi-question)
- [] 4.5 Frontend: Experience list + detail pages

Phase 5: Company Intelligence

- [] 5.1 Backend: company_service.py (CRUD + analytics computation)
- [] 5.2 Backend: company routes
- [] 5.3 Backend: Analytics recomputation (Celery task)
- [] 5.4 Frontend: Company list page

- [] 5.5 Frontend: Company intelligence page (full analytics view)

Phase 6: Resume Intelligence

- [] 6.1 Backend: AI resume_parser.py
- [] 6.2 Backend: resume routes (upload, insights)
- [] 6.3 Frontend: Resume upload (drag-drop) + insights display

Phase 7: AI Mock Interviews

- [] 7.1 Backend: AI interview_generator.py
- [] 7.2 Backend: AI followup_generator.py
- [] 7.3 Backend: mock_interview_service.py
- [] 7.4 Backend: interview routes (start, answer, complete)
- [] 7.5 Frontend: Interview config page
- [] 7.6 Frontend: Interview chat interface
- [] 7.7 Frontend: Interview history page

Phase 8: Evaluation + Replay

- [] 8.1 Backend: AI evaluator.py (explainable scoring)
- [] 8.2 Backend: evaluation routes
- [] 8.3 Frontend: Evaluation display (scores + reasons + radar chart)
- [] 8.4 Frontend: Interview replay page

Phase 9: Voice Engine

- [] 9.1 Backend: AI voice_engine.py (Whisper + Piper)
- [] 9.2 Backend: voice routes (transcribe, synthesize)
- [] 9.3 Frontend: VoiceRecorder component
- [] 9.4 Frontend: Voice mode integration in interview chat

Phase 10: Intelligence Engines

- [] 10.1 Backend: Weakness detection (post-evaluation hook)
- [] 10.2 Backend: AI recommendation_engine.py
- [] 10.3 Backend: readiness_service.py
- [] 10.4 Backend: knowledge_base_service.py (hybrid search)
- [] 10.5 Frontend: Weaknesses page
- [] 10.6 Frontend: Recommendations page
- [] 10.7 Frontend: Readiness page
- [] 10.8 Frontend: Knowledge base search page

Phase 11: Admin

- [] 11.1 Backend: admin routes (moderate, analytics)
- [] 11.2 Frontend: Admin layout + dashboard
- [] 11.3 Frontend: Admin company management
- [] 11.4 Frontend: Admin experience moderation
- [] 11.5 Frontend: Admin analytics page

Phase 12: Polish & Deploy

- [] 12.1 Error handling + loading states across all pages
- [] 12.2 Input validation (Zod on frontend, Pydantic on backend)
- [] 12.3 Activity logging middleware
- [] 12.4 Rate limiting
- [] 12.5 API documentation (auto-generated by FastAPI)
- [] 12.6 Unit tests (critical paths)
- [] 12.7 Deployment to Vercel + Render + Supabase
- [] 12.8 Production environment variables
- [] 12.9 Monitoring + logging setup

Section 24: Session Resumption Protocol

This section tells the AI assistant exactly how to resume work after a session break. Feed this entire PDF at the start of each session.

24.1 Resumption Steps

- > Step 1: Read this PDF completely to understand the full architecture and plan.
- > Step 2: Check the Progress Tracker (Section 25) to identify what's DONE vs PENDING.
- > Step 3: Run: 'find placement-intelligence-platform -name "*.py" -o -name "*.tsx" | head -50' to see current file state.
- > Step 4: Run: 'cat backend/app/main.py' to verify backend entry point exists.
- > Step 5: Run: 'ls frontend/src/app/' to verify frontend structure.
- > Step 6: Run: 'cd backend && alembic current' to check migration status.
- > Step 7: Identify the FIRST unchecked item in the Progress Tracker.
- > Step 8: Start building from that exact point. Do NOT rebuild completed items.
- > Step 9: After completing each task, acknowledge it as done.
- > Step 10: Follow the Build Order in Section 23 strictly. No skipping.

24.2 Key Rules for AI Assistant

- > NEVER start from scratch if files already exist. Always resume.
- > ALWAYS check existing code before writing new code.
- > FOLLOW the exact file paths in Section 4 (Directory Structure).
- > FOLLOW the exact DB schema in Section 5. Do not deviate.
- > FOLLOW the exact API contracts in Section 19. Do not change paths.
- > USE the exact tech stack versions in Section 3.
- > Every AI service must implement the BaseAIService interface.
- > Every evaluation score must have an explicit reason.
- > Interview follow-ups must be dynamic, never fixed flows.
- > Company analytics must weight recent data (90 days) higher.
- > All async operations go through Celery (AI processing, analytics recomputation).
- > Frontend uses TanStack Query for ALL data fetching.
- > Frontend uses Zustand for auth + interview session state only.
- > Frontend uses React Hook Form + Zod for ALL forms.

24.3 How to Update Progress

After completing tasks, the user should update the Progress Tracker. When asked 'where are we?', check the codebase and report: 1) Last completed phase, 2) Current phase, 3) Next task to do.

Section 25: Progress Tracker (Master Checklist)

This is the definitive checklist. Mark items [X] as they are completed. This tracker determines where to resume.

NOTE: Current Status: Phase 1 IN PROGRESS - Project scaffolding started, DB models partially created.

PHASE 1: Foundation

- ☐ 1.1 Backend directory structure created
- ☐ 1.2 requirements.txt with all dependencies
- ☐ 1.3 .env.example with all variables
- ☐ 1.4 core/config.py - Pydantic Settings
- ☐ 1.5 core/security.py - JWT + password utils
- ☐ 1.6 db/session.py - Async SQLAlchemy engine
- ☐ 1.7 models/user.py - User, Profile, Role, Permission, UserRole
- ☐ 1.8 models/company.py - Company, CompanyAnalytics
- ☐ 1.9 models/interview_experience.py - Experience, Question, Outcome
- ☐ 1.10 models/mock_interview.py - MockInterview, MockQuestion, Evaluation
- ☐ 1.11 models/intelligence.py - Weakness, Recommendation, ResumeData, ActivityLog
- ☐ 1.12 All Pydantic schemas (schemas/*.py)
- ☐ 1.13 app/main.py - FastAPI entry point
- ☐ 1.14 api/router.py - Main router with all sub-routers
- ☐ 1.15 Alembic init + initial migration
- ☐ 1.16 Frontend: Next.js project init
- ☐ 1.17 Frontend: Tailwind + Shadcn UI setup
- ☐ 1.18 Frontend: Layout components (Sidebar, Header)
- ☐ 1.19 Frontend: TypeScript types (types/index.ts)
- ☐ 1.20 Frontend: API client (lib/api.ts)
- ☐ 1.21 Frontend: Supabase client (lib/supabase.ts)
- ☐ 1.22 Docker compose (Postgres + Redis)

PHASE 2: Authentication

- ☐ 2.1 Backend: auth_service.py
- ☐ 2.2 Backend: auth routes
- ☐ 2.3 Backend: user routes
- ☐ 2.4 Frontend: Auth store (Zustand)
- ☐ 2.5 Frontend: Login page
- ☐ 2.6 Frontend: Register page
- ☐ 2.7 Frontend: AuthGuard HOC
- ☐ 2.8 Frontend: Profile page

PHASE 3: Dashboard

- ☐ 3.1 Backend: dashboard_service.py

- [] 3.2 Backend: dashboard route
- [] 3.3 Frontend: Dashboard page (all sections)
- [] 3.4 Frontend: TrendGraph component

PHASE 4: Interview Experiences

- [] 4.1 Backend: experience_service.py
- [] 4.2 Backend: experience routes
- [] 4.3 Backend: AI knowledge_extractor.py
- [] 4.4 Frontend: Experience form (multi-question)
- [] 4.5 Frontend: Experience list + detail

PHASE 5: Company Intelligence

- [] 5.1 Backend: company_service.py + analytics
- [] 5.2 Backend: company routes
- [] 5.3 Backend: Celery analytics task
- [] 5.4 Frontend: Company list page
- [] 5.5 Frontend: Company intelligence page

PHASE 6: Resume Intelligence

- [] 6.1 Backend: AI resume_parser.py
- [] 6.2 Backend: resume routes
- [] 6.3 Frontend: Resume upload + insights

PHASE 7: AI Mock Interviews

- [] 7.1 Backend: AI interview_generator.py
- [] 7.2 Backend: AI followup_generator.py
- [] 7.3 Backend: mock_interview_service.py
- [] 7.4 Backend: interview routes
- [] 7.5 Frontend: Interview config page
- [] 7.6 Frontend: Interview chat interface
- [] 7.7 Frontend: Interview history

PHASE 8: Evaluation + Replay

- [] 8.1 Backend: AI evaluator.py
- [] 8.2 Backend: evaluation routes
- [] 8.3 Frontend: Evaluation display
- [] 8.4 Frontend: Interview replay

PHASE 9: Voice Engine

- [] 9.1 Backend: voice_engine.py (Whisper + Piper)
- [] 9.2 Backend: voice routes
- [] 9.3 Frontend: VoiceRecorder component
- [] 9.4 Frontend: Voice mode integration

PHASE 10: Intelligence Engines

- [] 10.1 Backend: Weakness detection
- [] 10.2 Backend: AI recommendation_engine.py
- [] 10.3 Backend: readiness_service.py
- [] 10.4 Backend: knowledge_base_service.py
- [] 10.5 Frontend: Weaknesses page
- [] 10.6 Frontend: Recommendations page
- [] 10.7 Frontend: Readiness page
- [] 10.8 Frontend: Search page

PHASE 11: Admin Panel

- [] 11.1 Backend: admin routes
- [] 11.2 Frontend: Admin layout + dashboard
- [] 11.3 Frontend: Admin companies
- [] 11.4 Frontend: Admin experience moderation
- [] 11.5 Frontend: Admin analytics

PHASE 12: Polish & Deploy

- [] 12.1 Error handling + loading states
- [] 12.2 Input validation (all forms)
- [] 12.3 Activity logging middleware
- [] 12.4 Rate limiting
- [] 12.5 API documentation review
- [] 12.6 Unit tests
- [] 12.7 Deploy to Vercel + Render + Supabase
- [] 12.8 Production env vars
- [] 12.9 Monitoring setup

Final Notes

This document is the COMPLETE blueprint for the Placement Intelligence Platform. It contains every detail needed to build the platform from scratch or resume at any point.

Key Principles

- > Every interview submitted improves the platform for all students.
- > The moat is data (real experiences), not AI.
- > AI scores must always be explainable with specific reasons.
- > Interview follow-ups must be dynamic and adaptive.
- > Recent data (90 days) is weighted higher for company intelligence.
- > All AI services are modular and independently replaceable.
- > Design for multi-tenancy from day one.
- > The platform should become the operating system for placement preparation.

Build something that becomes smarter every single day.